## Lecture 12 - Extensible Query Optimization

- Surajit's paper mentions two different schools of thought for extensibility: Starburst and Volcano/Cascades
- Today, will focus on the Volcano/Cascades story from a group of Microsoft folks
  - BTW, here's a slightly older version of the manuscript; this has simplified pseudocode and examples in many places (that doesn't catch all corner cases, but maybe that's ok!)

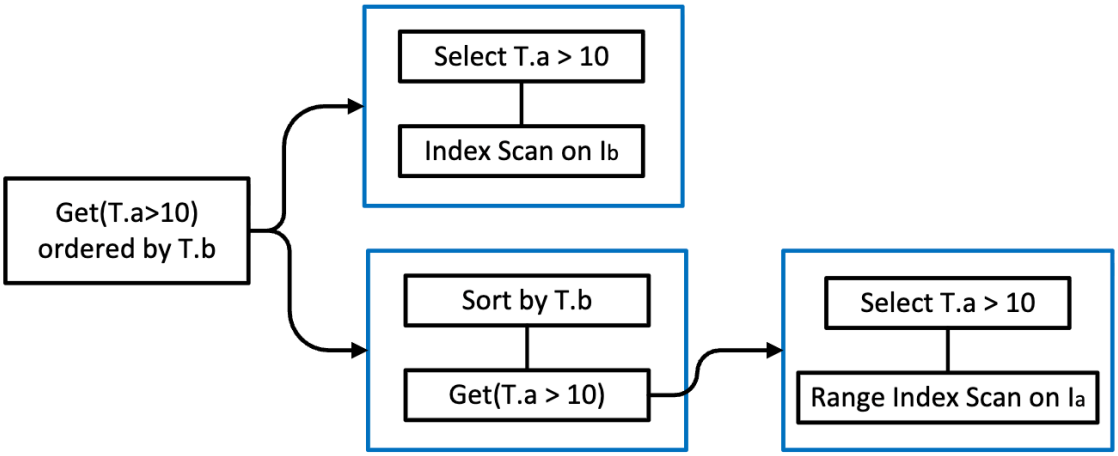
Some History and Background

- Volcano/Cascades story begins with Goetz Graefe who started as DeWitt PhD student on the Exodus project.
- He did a series of rule-driven extensible optimizer generators:
  - Exodus Optimizer Generator at Wisconsin (with DeWitt),
  - Volcano Optimizer Generator at CU Boulder, and
  - Cascades at Portland State University. Graefe went on to join the SQL Server team early in the history of Microsoft, and made it his mission for Cascades to succeed.
  - The Volcano paper is the one of these three that's easiest to read.
  - The Cascades paper is hard to read.
  - The present paper is the first good explanation of Cascades within MSR
- Cascades ideas also thrived outside MS:
  - Two open-source efforts:
    - Apache Calcite (designed by Julian Hyde during the Hadoop years)
    - Orca (designed by a SQL Server alumnus, Florian Waas)
  - Oracle switched from a Selinger-style optimizer to Cascades at some point.
- The Volcano system, is also credited with two important contributions:
  - the iterator model of operators (which was known, but formalized by Volcano)
  - and the notion of the "exchange" operator documented in another important paper
- Iterator "pull-based" model of operators:
  - Each operator has *Open*, *GetNext*, *Close* methods
  - *Open*: initialize state
  - *GetNext*: produce next output, may recursively call child operator(s)
  - While a nice clean abstraction, doesn't work as well in practice on modern CPUs:
    - many function calls just to propagate one tuple up the plan
    - instead, *vectorization* operates on and generates a batch at a time, also leveraging SIMD instructions, pioneered by MonetDB
      - works especially well with columnar formats; will look at later
      - can also generate custom optimized code using a framework like LLVM
- Exchange operator model
  - Basic idea: don't generate custom "parallel" physical implementations of each operator, just encapsulate the parallelism via a special *exchange* operator
  - Exchange is an adapter between:
    - the **demand-driven iterator world** inside one process, and
    - the **parallel/data-transfer world** across processes

### The Volcano Algorithm

- Separates out Logical properties and Physical properties
  - Logical: schema, cardinality
  - Physical: sort order, indexes, compression, parallel partitioning, etc.
    - Generalizes the notion of "interesting orders" in System R

- Logical algebra and Physical algebra
  - E.g. join vs hashjoin
  - Certain operators are **Enforcers**: they have no logical analogue, they exist to enforce a physical property
    - E.g. Sort in System R enforces interesting orders
    - `SELECT * FROM T WHERE T.a > 10 ORDER BY T.b LIMIT 3`




- Search in two dimensions:
  - *Transformation* rules: Logical <-> Logical equivalences ("source to source" transformations)
  - *Implementation* rules: Logical <-> Physical equivalences ("lowering")
  - Defined using two methods: `CheckPattern` and `Transform`
    - former checks applicability, latter transforms
- Contrasting with Starburst
  - Starburst had two rule-driven extensible optimizers: one for Query Rewriting, one for Cost-Based Optimization
  - Query Rewriting for canonicalizations that almost always help.
    - E.g. flatten subqueries
    - E.g. copy predicates around (Yannakakis, "magic sets") TODO
    - E.g. redundant join elimination ("RedJel")
  - Cost-based optimization for the rest a la Selinger, but with extensible DP transformation "grammar"
  - IBM products still work this way, seems to be about as good as Cascades
- Search strategy
  - Selinger/System R is bottom-up. Starburst's rule-driven optimizer was also bottom-up.
  - Cascades is top-down (goal oriented) with memoization
  - No clear winner; much ink has been spilled over top-down vs bottom-up.
  - Both can work with extensible rules.
  - In terms of efficiency, they are probably isomorphic (see the datalog literature), but the pruning is much more obvious in "top-down".

Details

- Top-down optimization:
  - Given a goal (say R join S join T), recursively solve subgoals (say R join S, S join T, R join T…)
  - To avoid redundancy, save what you find in a lookup ("memo") table
  - If you have an existing candidate solution for a goal, you can use its cost to prune search for subgoals
- Volcano's *memo* table
  - *group*: equivalent expressions
  - may reference "child" groups

- *group expression*: can be logical or physical
  - per expression track logical and physical properties
- during **generation phase**, adds new groups and group logical expressions as we find them
- during **cost analysis phase**, cache best physical plan for each logical expression and its required physical properties
  - up to some cost limit!
  - pruning is nice in top-down search:
    - if we know a cost c of a super-expression is at most c, we can constrain search of subexpressions to be c or less

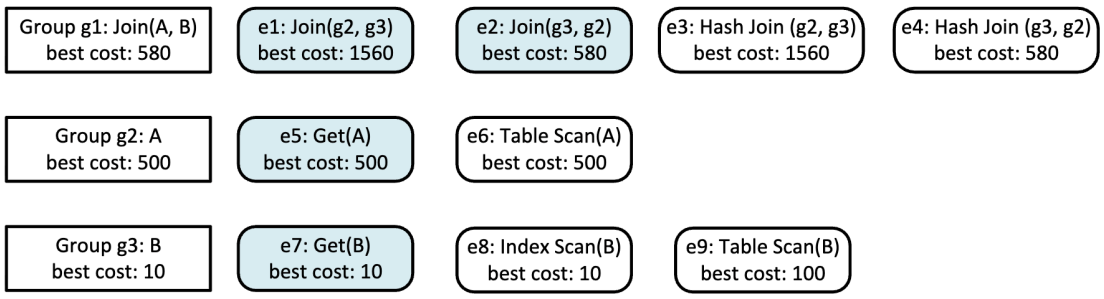



**Figure 2.3:** An example memo of $A \bowtie B$, where an index is available on table $B$. Logical expressions are colored in blue. Physical expressions and groups are annotated with the cost of the corresponding best plans.

- Algorithm:
  - Start with any legal logical plan from the parser
  - `GenerateLogicalExpression`: DFS. // Generation Phase
    - Generate equivalent plans for a given expression after generating equivalent plans for its child expressions (recursively - depth-first)
    - after recursion pops, `MatchTransRule` adds all equivalent logical expressions at this level to memo
    - inside `MatchTransRule`, if we generate a new logical expression in memo we invoke `GenerateLogicalExpression` recursively.
    - 2 logical expressions based on a single transformation are connected as neighbors (see below)
    - Pseudocode only for reference:

```
function GenerateLogicalExpr(LogExpr, Rules)          ▷ Generation phase
    for Child in inputs of LogExpr do
        if Group(Child) ∉ Memo then
            GenerateLogicalExpr(Child)                 ▷ Update memo
    MatchTransRule(LogExpr, Rules)
function MatchTransRule(LogExpr, Rules)   ▷ Apply transformation rules
    for rule in Rules do
        if rule matches LogExpr then
            NewLogExpr ← Transform(LogExpr, rule)      ▷ Update memo and record the neighbors
            GenerateLogicalExpr(NewLogExpr)            ▷ Only invoke if NewLogExpr is not previously in memo
```

  - `FindBestPlan` // Cost Analysis
    - Input: Initial logical plan, physical properties of output (e.g. sort order), and a pruning limit (initial call is infinity)
    - Transformations called `moves` (as in AI game search?)
      - Logical, Physical, or Enforcers.
      - Sort next moves via `promise`, a heuristic that hopefully finds low pruning limits early on!
        - Can also prune heuristically (eg bushy plans have promise 0)

    - If next move is logical, we’re looking at neighbors in logical expression space
    - Physical transformations result (recursively) in costed plans
    - Enforcers result in new physical properties, kicking off recursive search of alternatives to satisfy the new logical/physical pairing
    - `BestPlan` and `BestCost` are maintained as we go.
- Most additional book-keeping to record "in progress" so that we don't restart from scratch when we revisit an expression
- Pseudocode only for reference:

```
function FindBestPlan(LogExpr, PhyProp, Limit)   ▷ Cost analysis phase
    if (LogExpr, PhyProp) in the Memo then                ▷ Check duplicate
        Plan, Cost ← LookUpBestPlan(LogExpr, PhyProp)
        if Cost ≠ null and Cost ≤ Limit then
            return Plan, Cost
        else
            return null, null  ▷ No such plan exists based on previous attempts
    MarkInProgress(LogExpr, PhyProp)          ▷ Optimize a new expression
    Moves ← GetAllMoves(LogExpr, PhyProp)       ▷ Generate all applicable
transformations, potentially with heuristics or guidance
    SortedMoves ← SortMovesByPromise(Moves)          ▷ Order by promise
    BestPlan ← null
    BestCost ← ∞
    for m ∈ SortedMoves do                                ▷ Apply each move
        if m is a transformation rule then
            for NewLogExpr in GetNeighbors(LogExpr, m) do       ▷ Retrieve
equivalent alternative logical expressions populated in the generation phase
                if !InProgress(NewLogExpr, PhyProp) then
                    FindBestPlan(NewLogExpr, PhyProp, Limit)  ▷ Can fail if
such a plan is not found
        else if m is a physical implementation rule then
            TotalCost ← DeriveCost(LogExpr, m)       ▷ Cost the root operator
            Subplans ← []
            for Child in inputs of LogExpr do
                ChildProp ← DerivePhyProp(LogExpr, PhyProp, Child)
                Plan, Cost ← FindBestPlan(Child, ChildProp, Limit − Total
Cost)                                  ▷ Can fail if such a plan is not found
                TotalCost ← TotalCost + Cost
                Subplans.Add(Plan)
            Plan ← CreatePlan(m, SubPlans)
```

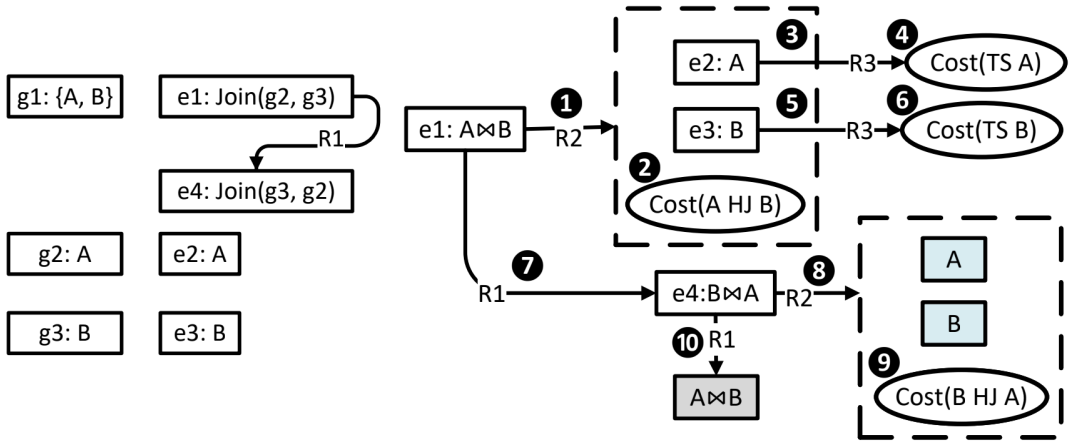


```
            UpdatePlan(BestPlan, BestCost, Plan, TotalCost) ▷ Update memo
        else if m is an enforcer then              ▷ Special handling of enforcers
            Cost ← DeriveCost(LogExpr, m)                  ▷ Cost the enforcer
            NewProp ← GetPhyProp(LogExpr, PhyProp, m)
            if !InProgress(LogExpr, NewProp) then
                Plan, Cost ← FindBestPlan(LogExpr, NewProp, Limit − Cost)
▷ Can fail if such a plan is not found
                UpdatePlan(BestPlan, BestCost, Plan, Cost)   ▷ Update memo
        Limit ← min(Limit, BestCost)                  ▷ Update the cost limit
    Memo.Add(LogExpr, PhyProp, BestPlan, BestCost)          ▷ Cache result
    return BestPlan, BestCost
```

- Nice example if all we have is Join Commutativity, Associativity, Hashjoin, and TableScan

- Join commutativity (R1): A ⋈ B → B ⋈ A
- Hash join (R2): A ⋈ B → HashJoin(A, B)
- Table scan (R3): A → TableScan(A)
- Example

**Figure 2.4:** Example of how the Volcano optimizer explores the search space for optimizing $A \bowtie B$




**(a)** The memo of groups and equivalent logical expressions after the generation phase

**(b)** The top-down dynamic programming in cost analysis phase. The derived expression is marked in blue if it has already been optimized; and it is marked in grey if it is a duplicate expression.

- Steer the rules with *promise* and also "*Guidance*"
  - "*Guidance specifies the rules that are considered to be applied for a given expression, and thereby can constrain the search space. Guidance can include activating only a subset of rules for a more focused search space, or limiting the depth of the search by deactivating rules when the number of transformations from the original query to the current expression reaches a threshold*"
- Overall:
  - DP is achieved top-down via the memo table
    - Every optimal plan is constructed once out of optimal subplans, recursively
  - Pruning can be very effective:
    - if we're lucky, need not explore all candidate subplans, unlike Selinger
    - Exercise for home: can Selinger be enhanced to do this?
    - can even choose to seed memo with an initial plan/cost based on a heuristic optimizer!
  - Totally rule-driven and extensible!

**To Cascades!**

- Main drawback of Volcano is efficiency of search due to the split between logical and physical plans.
  - This is artificial, and leads Volcano to enumerate the entire logical space during Generation phase, before pruning/guidance in the Cost Analysis phase. Means:
    - No cost-based Pruning during Generation. Generation may take too long!
    - Guidance is controlled on a *rule* level during Cost Analysis, within a group of logical expressions. Guidance is not applied at logical expression generation level, limiting heuristics prior to Cost Analysis
      - (e.g. "no bushy trees").
- Does not do recursive invocations, instead stages work in the form of tasks in a LIFO stack
  - Avoids sequential optimization of independent expressions - can parallelize
- Transformation Rules in Cascades
  - Remove distinction between physical/logical ops
  - Similarly remove distinction between logical/implementation rules - just rules

    - Addition of 2 kinds of rules
        - Enforcer rules to change physical props.
            - Compared to Volcano, which ads Enforcers ad-hoc, here unified into rules
            - Applied when the physical property is needed, rather than arbitrary
        - Function rules: transforms an expression into many alternatives in one call
- Some subtle ideas:
    - In Cascades' memo:
        - Entry for each group: contains list of expressions
        - Within each expression: bitmap of which rules have been applied so far
    - Once we apply a transform, we can remove it from guidance if reapplication just generates the inverse (e.g. commutativity of join)
    - Can "stage" the optimizer by changing guidance incrementally
    - Simplification Rules
        - Rules that almost always help, can be pre-processed.
        - Can get the Starburst "rewrite-then-explore" optimizations implemented that way